

多模态 Agent Post-Training 项目

完整分析报告

2026 年 5 月 24 日

摘要

本项目以 Qwen2.5-VL-3B/7B 为基座，训练 VLM 在发票核验、图表计算、图文一致性等场景下严格遵循 `vision_parse` → `retrieve_docs/python_exec` → `final_answer` 的多步工具调用流程。本文从代码架构、问题演化、优化历程、失败分析、当前瓶颈、后续方向六个维度进行系统性梳理。核心发现：(1) 单阶段 SFT 在同时优化格式纪律、工具选择、答案质量、拒答边界时存在根本性多目标冲突；(2) 项目中唯一有效的优化全部来自多模态层面（真实 VLM 替代 mock、3B 升 7B），Agent 层面的增量修补（DPO、增强数据、微调）全部失败；(3) 7B 在文档理解上碾压 3B，但对训练数据偏移的敏感度呈数量级增加；(4) 当前最优为 V4(3B) 和 V7B-clean(7B)，后续 RL 优化的真正空间在 7B 的工具纪律修复。

目录

1	项目定义	1
1.1	任务描述	1
1.2	代码架构	1
1.3	评测体系	2
2	分阶段历程	3
2.1	阶段 0: 基础建设 (V2 之前)	3
2.2	阶段 1: Mock 数据时代 (V2)	3
2.3	阶段 2: V3 过度工具调用	3
2.4	阶段 3: 真实 VLM 数据 (V4) —全项目最大单次提升	3
2.5	阶段 4: 数据增强 (V5/V6/V8) —全部失败或持平	4
2.6	阶段 5: DPO (三次尝试) —全部崩溃	4
2.7	阶段 6: 7B 迁移 (V7B-clean) —文档理解质的飞跃	4
2.8	阶段 7: RL/GRPO (当前迭代)	4
3	核心问题总结	4
4	优化上限分析	5
4.1	3B 天花板	5
4.2	7B 天花板	5
4.3	RL 理论上限	5

1 项目定义	2
5 当前最优模型	6
6 RL 训练命令	6
7 后续方向	6

1 项目定义

1.1 任务描述

给定一张图片和一个自然语言问题，VLM 需按以下流程完成任务：

1. 判断任务类型（发票核验/图表计算/图文一致性/品牌识别/简单描述）
2. 如需视觉信息，调用 `vision_parse(mode)` 进行 OCR/图表读数/图像描述
3. 如需规则依据，调用 `retrieve_docs(query)` 检索规则文档
4. 如需数学计算，调用 `python_exec(code)` 完成计算
5. 输出包含依据的 `final_answer` 或合理拒答

核心约束：每轮对话模型只能输出一个 `<tool_call>` 或一个 `<final_answer>`，不得同时输出多个动作或裸文本。这一格式约束是整个项目的阿喀琉斯之踵。

1.2 代码架构

文件	功能
agent/runtime.py	多轮 Agent 执行引擎—管理消息循环、enforce 拦截、工具调用
agent/tools.py	三个工具实现— <code>vision_parse</code> (真实 VLM/mock)、 <code>retrieve_docs</code> (规则检索)、 <code>python_exec</code> (安全 eval)
agent/vlm.py	Qwen-VL 模型封装—加载、LoRA 注入、mock 回退
agent/parser.py	输出解析—正则提取 <code>tool_call</code> 或 <code>final_answer</code> ，容错缺闭合标签
agent/prompts.py	System prompt—定义工具调用格式规范
train/train_sft_lora.py	SFT LoRA 训练— <code>expand_assistant_turns</code> + <code>prefix/full collator</code>
train/train_dpo_lora.py	DPO 偏好训练— <code>last-turn log-prob</code> 优化（三次崩溃，已弃用）
train/train_rl_custom.py	自研 RL 训练—REINFORCE+KL，无 TRL 依赖（当前迭代中）
eval/run_vlm_eval.py	评测主脚本—ToolHit, ToolOrder, Format, Finished, Keyword, Refusal
eval/run_seed_eval.py	早期简化评测（不依赖真实 VLM）
scripts/build_agent_train_real_v3.py	合并 SFT+ 外部数据集，控制上采样和 <code>recovery</code> 参数
scripts/rebuild_seed_with_real_vlm.py	用真实 VLM 重建训练数据中的 <code>vision_parse</code> 结果

<code>scripts/real_vision_parse.py</code>	共享真实 VLM 推理模块
<code>scripts/convert_chartqa/docvqa/cord_to_sft.py</code>	外部数据集转换为 Agent SFT 格式
<code>scripts/build_augmented_v5_data.py</code>	数据增强—多样问法、推理链、纠错轨迹
<code>scripts/build_hard_cases_from_eval.py</code>	从评测错误生成 SFT 修正样本和 DPO 偏好对
<code>scripts/build_tool_decision_samples.py</code>	工具决策样本—直接回答 +refusal+tool
<code>scripts/build_refusal_samples.py</code>	用 7B 真实 VLM 生成防幻觉样本
<code>scripts/tag_vlm_errors.py</code>	评测结果自动错误标注

1.3 评测体系

50 条评测样本,分 5 类:document_validation(15)、image_text_consistency(12)、chart_calculation(10)、uncertainty_refusal(8)、direct_vqa(5)。

6 项指标:

1. Tool Hit: $\text{gold_tools} \subseteq \text{pred_tools}$
2. Tool Order Hit: gold_tools 顺序一致
3. Format Valid: 无 parse_error
4. Finished: 正常结束 (未超 max_steps)
5. Answer Keyword Hit: gold_keywords 全部出现在答案中
6. Refusal Correct: should_refuse 与 is_refusal 一致

2 分阶段历程

2.1 阶段 0: 基础建设 (V2 之前)

问题: 模型不会按流程调工具。

做法:手工构造 50 条 seed 数据,每条包含完整的多轮工具调用轨迹。通过 build_first_tool_sft.py、build_second_tool_sft.py、build_sft_train.py 三个脚本拆分为 first_tool、full_trace、second_tool 三类训练样本。

2.2 阶段 1: Mock 数据时代 (V2)

问题: 工具命中率只有 50%, 模型经常跳过第二工具。

做法: 在 runtime.py 实现 enforce_required_tools—关键词匹配判断是否需要第二工具, 漏了就拦截 final_answer 并注入纠正提示。加入 ChartQA 外部数据 (446 条)。

关键代码 (runtime.py):

```
required_tool = self._required_tool_after_vision(question, called_tools)
if self.enforce_required_tools and required_tool:
    messages.append({"role": "user",
                     "content": self._missing_required_tool_prompt(required_tool)})
    continue
```

效果: 工具命中率从 50% 升至 100%。但这是 runtime 兜底, 不是模型学会的。

2.3 阶段 2: V3 过度工具调用

问题: 模型在不需要工具的场景也强行调工具。根因是 recovery 变体教模型”宁可多调不能漏调”, ChartQA/CORD 上采样 5x 过度放大第二工具信号。

做法: recovery_repeat 2→1, python_exec/retrieve_docs repeat 5→2, recovery 提示词对齐 runtime。

2.4 阶段 3: 真实 VLM 数据 (V4) —全项目最大单次提升

问题: 训练数据中 vision_parse 结果是人工 mock 的, 外部数据集甚至用占位文本。模型从未见过真实 VLM 输出分布。

做法:

- 新建 rebuild_seed_with_real_vlm.py: 加载真实 Qwen2.5-VL-3B, 对 46 张种子图片逐张做 vision_parse, 替换 mock 结果
 - 新建 real_vision_parse.py: 共享真实 VLM 推理模块
 - 修改 convert_chartqa/docvqa/cord_to_sft.py: 支持-use_real_vlm 标志
 - 797 张图片全部完成真实 VLM 处理
 - 外部图片 Resize 到 max 1024px (DocVQA 最大 1.8MB、CORD 最大 7.4MB 导致 OOM)
- 效果: V4 成为 3B 最优模型。这是全项目最大单次提升。

2.5 阶段 4: 数据增强 (V5/V6/V8) —全部失败或持平

做法:

- V5: build_augmented_v5_data.py 生成 818 条增强数据 (632 条多样问法 +80 条推理链 +42 条纠错轨迹 +6 条 refual+8 条直接回答)
 - V6: 修复 refual 模板 (从假模板换为真实 VLM 输出)
 - V8: build_tool_decision_samples.py 生成 53 条工具决策样本
- 结果: V5 持平 (82%)、V6 退步 (70%)、V8 大幅退步 (50%)。

根因: 单阶段 SFT 需要同时优化四个互相矛盾的目标—格式纪律、工具选择、答案质量、拒答边界。这些在梯度空间中是竞争关系。任何针对单一目标的修补都会打破平衡。

2.6 阶段 5: DPO (三次尝试) —全部崩溃

现象:

- DPO 单轮 (64 对): tool hit 100%→60%
- DPO 多轮 (13 对): tool hit→14%, 输出空字符串
- SFT 微调 (13 条): tool hit→22%

根因: train_dpo_lora.py 的 compute_log_prob 只优化最后一条 assistant turn。偏好数据是单轮的 user→final_answer。模型可以在降低 DPO loss 的同时完全抛弃多轮格式约束。

2.7 阶段 6: 7B 迁移 (V7B-clean) —文档理解质的飞跃

问题: 3B 的视觉理解能力是瓶颈。

做法：用与 V4 完全相同的 1133 条数据，换 Qwen2.5-VL-7B-Instruct。经过 rank 搜索：r=16 崩溃 (14% tool)、r=32 成功 (92%)、r=64 崩溃 (16%)。

关键发现：7B 对训练数据分布偏移的敏感度比 3B 高一个数量级。加 9 条 refusal 样本 (0.8% 的数据改动) 就能让工具命中从 92% 崩到 16%。

效果：document_validation 10/15→15/15, image_text_consistency 9/12→12/12。

2.8 阶段 7: RL/GRPO (当前迭代)

经历了四轮迭代：

1. 用 TRL 的 GRPOTrainer—失败 (Qwen2.5-VL RoPE 索引兼容性 bug)
2. 手写 GRPO, ratio 分母 $=\pi_{ref}$ (SFT) —3B 跑 11 轮 reward 不动, 模型被困在 SFT 局部最优
3. 修正 ratio 分母 $=\pi_{old}$, 用跨轮缓存—loss 爆炸 (1.58→4.51), 根因是缓存按位置匹配但文本每轮不同, ratio 成纯噪声
4. 放弃跨轮 ratio, 改用纯 REINFORCE+KL (ratio=1), kl_coef↑0.2, lr↓1e-6—当前版本

3 核心问题总结

#	问题	根因	解决	效果
1	Mock 数据 vs 真实 VLM 不匹配	训练用人工 OCR 结果	rebuild_seed_with_全项vln.py 重建 797 张图	最大提升
2	大图 OOM	Qwen-VL 动态分辨率	Resize 到 max 1024px	训练可行
3	SFT 多目标冲突	格式 + 工具 + 答案 + 拒答抢一个 loss	未解决	所有增量补丁失败
4	DPO 格式崩溃	DPO 只优化 last-turn	放弃 DPO	三次崩溃
5	7B 数据极度敏感	大模型捷径学习	只用纯 V4 数据, 不添加额外样本	r=32 成功
6	GRPO 跨轮 ratio 噪声	缓存按位置匹配, 文本每轮不同	改为 ratio=1 (纯 REINFORCE)	待验证
7	7B 双模型 GPU OOM	两个 7B≈30GB	-ref_on_cpu flag	待验证
8	3B reward 到天花板	V4 已 100%tool	GRPO 对 3B 无效	放弃 3B, 聚焦 7B

4 优化上限分析

4.1 3B 天花板

V4(3B) 的工具纪律已达 100%，文档理解受限于 3B OCR 质量。群体 rollout reward 同质化，GRPO 无优化信号。3B 已到极限。

4.2 7B 天花板

V7B-clean(7B) 文档理解满分 (15/15+12/12)，但工具纪律 92%。品牌识别存在真实的漏调 (refusal_002 的”Vans” 幻觉)，reward 有区分度。

7B 是 RL 的真正目标。

4.3 RL 理论上限

1. REINFORCE 固有方差问题: 40 样本 $\times$ 4rollout 的梯度噪声大，需 1000+ rollout 才能稳定收敛
2. KL 锚定 SFT 限制了探索范围—tradeoff 安全性 vs 优化空间
3. reward 函数设计决定了优化方向: 工具奖励 (1.0) > 答案奖励 (0.5) > 拒绝奖励 (0.5) > 惩罚 (-2.0~-0.3)

5 当前最优模型

指标	V4 (3B)	V7B-clean (7B)
Tool Hit	50/50	46/50 (无 enf 49/50)
Keyword	38/50	43/50
文档核验	10/15	15/15
图文一致性	9/12	12/12
品牌识别	8/8 tool	4/8 tool
数据敏感性	低	极高
RL 优化空间	无 (天花板)	有 (92%→100% tool)

表 3: 两个最优模型的能力互补

6 RL 训练命令

3B (验证用，两个模型都在GPU)

```
python train/train_rl_custom.py \
  --model_name .../Qwen2.5-VL-3B-Instruct \
  --adapter_name .../lora_agent_real_v4 \
  --output_dir .../lora_grpo_v1 --bf16
```

7B (主跑, ref放CPU避免OOM)

```
python train/train_rl_custom.py \  
    --model_name .../Qwen2.5-VL-7B-Instruct \  
    --adapter_name .../lora_agent_v7b_clean \  
    --output_dir .../lora_grpo_v2_7b \  
    --ref_on_cpu --lr 1e-6 --kl_coef 0.2
```

7 后续方向

1. 7B + RL 优化: 修复品牌识别 (92%→100% tool), 消除”Vans” 幻觉
2. reward 函数增强: 对 parse_error 加重罚分 (-2→-5), 强化格式保护
3. 增大 rollout 数量: K=4→8, 增加采样多样性
4. 跨模型蒸馏: V4 的工具纪律 →V7B, 融合两者优势
5. 模型规模工具学习动力学研究: 为什么 7B 文档理解强但工具纪律弱? 为什么 7B 对数据偏移更敏感?